

A photograph of four students in a library setting. A young woman with long dark hair is on the left, smiling. Next to her is a young man with dark hair, also smiling. To his right is a young woman with glasses and dark hair, looking towards the right. On the far right is a young man with dark hair, seen from the back/side. They are gathered around a table with a laptop and several open books. The background is filled with bookshelves. A semi-transparent blue diagonal band runs across the image, and a semi-transparent red horizontal band is at the bottom.

Java 25

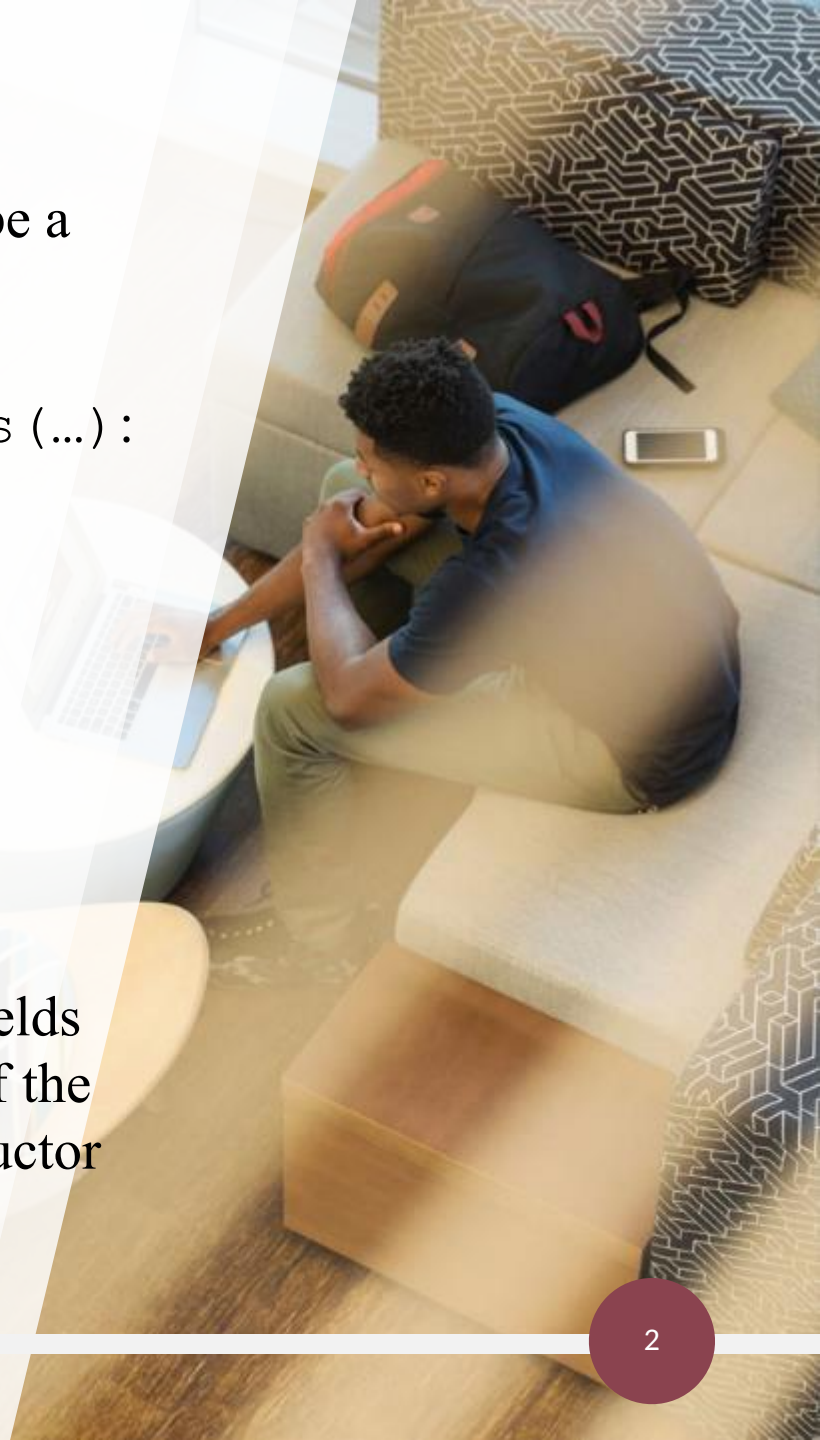
Flexible Constructor Bodies

Flexible Constructor Bodies

- Previously (pre-Java 25), the very first statement in a constructor had to be a call to `super (...)` or `this (...)`.
- Now, we have code that can appear before the call to `super (...)` or `this (...)`:

```
class C extends Object{  
    C(...) {  
        // prologue code (limited)  
        // super(...); or this(...)  
        // epilogue code  
    }  
}
```

- In the prologue area, we can now validate inputs and initialise instance fields **before** chaining to another constructor, thereby establishing the *integrity* of the object. Our code is now easier to follow as validation logic is in the constructor itself (and not in helper methods).



Flexible Constructor Bodies

- Additionally, parent class constructor calls to overridable methods are now safer in one crucial aspect - the subclass can now initialise instance variables before `super (...)`, reducing the chance of `null` values.
- That said, even though we can initialise instance fields in the prologue area, we cannot call instance methods or read instance fields - `super (...)` must run first (because, the object is only partially constructed and as a result it is not safe to use `this` yet).
- As before, if there is no explicit `super (...)` or `this (...)`, Java inserts `super ()`; as normal (if the superclass has an accessible no-arg constructor).

